11.4.1 Component-Level Design for WebApps

The boundary between content and function is often blurred when Web-based systems and applications (WebApps) are considered. Therefore, it is reasonable to ask: What is a WebApp component?

In the context of this chapter, a WebApp component is (1) a well-defined cohesive function that manipulates content or provides computational or data processing for an end user or (2) a cohesive package of content and functionality that provides the end user with some required capability. Therefore, component-level design for WebApps often incorporates elements of content design and functional design.

Content design at the component level focuses on content objects and the ways they may be packaged for presentation to a WebApp end user. The formality of content design at the component level should be tuned to the characteristics of the WebApp to be built. In many cases, content objects need not be organized as components and can be manipulated individually. However, as the size and complexity (of the WebApp, content objects, and their interrelationships) grows, it may be necessary to organize content in a way that allows easier reference and design manipulation.[6] In addition, if content is highly dynamic (e.g., the content for an online auction site), it becomes important to establish a clear structural model that incorporates content components.

A good example of a component that might be part of an e-commerce WebApp is the "shopping cart." A shopping cart provides a convenient way for e-commerce customers to store and review their selected items prior to checking out. They can then pay for their selections with a single transaction at the end of their e-commerce session. A carefully designed shopping cart can be reused in several Web store applications by simply editing its content model.

WebApp functionality can be delivered as a series of components developed in parallel with the information architecture to ensure consistency. The shopping cart component described previously contains both content and algorithmic elements. You begin by considering both the requirements model and the initial information architecture. Next, you examine how functionality affects the user's interaction with the application, the information that is presented, and the user tasks that are conducted.

During architectural design, WebApp content and functionality are combined to create a functional architecture. A *functional architecture* is a representation of the functional domain of the WebApp and describes the key functional components in the WebApp and how these components interact with each other.

11.4.2 Component-Level Design for Mobile Apps

Mobile apps are typically structured using multilayered architectures, including a user interface layer, a business layer, and a data layer. If you are building a mobile app as a thin Web-based client, the only components residing on a mobile device are those required to implement the user interface. Some mobile apps may incorporate the components required to implement the business and/or data layers on the mobile device, subjecting these layers to the limitations of the physical characteristics of the device.

---

6 Content components can also be reused in other WebApps.

Considering the user interface layer first, it is important to recognize that a small display area requires the designer to be more selective in choosing the content (text and graphics) to be displayed. It may be helpful to tailor the content to a specific user group(s) and display only what each group needs. The business and data layers are often implemented by composing Web or cloud service components. If the components providing business and data services reside entirely on the mobile device, connectivity issues are not a significant concern. Intermittent (or missing) Internet connectivity must be considered when designing components that require access to current application data that reside on a networked server.

If a desktop application is being ported to a mobile device, the business-layer components may need to be reviewed to see if they meet nonfunctional requirements (e.g., security, performance, accessibility) required by the new platform. The target mobile device may lack the necessary processor speed, memory, or display real estate. The design of mobile applications is considered in greater detail in Chapter 13.

An example of a component in a mobile application might be the single-window full-screen user interface (UI) typically designed for phones and tablets. With careful design it may be possible to allow the mobile app to sense the display characteristics of the mobile device and adapt its appearance to ensure that text, graphics, and UI controls function correctly on many different screen types. This allows the mobile app to function in similar ways on all platforms, without having to be reprogrammed.

11.4.3 Designing Traditional Components

The foundations of component-level design for traditional software components were formed in the early 1960s and were solidified with the work of Edsger Dijkstra ([Dij65], [Dij76b]) and others (e.g., [Boh66]). A traditional software component implements an element of processing that addresses a function or subfunction in the problem domain or some capability in the infrastructure domain. Often these traditional components are called functions, modules, procedures, or subroutines. Traditional components do not encapsulate data in the same way that object-oriented components do. Most programmers make frequent use of function libraries and data structure templates when developing new software products.

In the late 1960s, Dijkstra and others proposed the use of a set of constrained logical constructs from which any program could be formed. The constructs emphasized "maintenance of functional domain." That is, each construct had a predictable logical structure and was entered at the top and exited at the bottom, enabling a reader to follow procedural flow more easily.

The constructs are sequence, condition, and repetition. *Sequence* implements processing steps that are essential in the specification of any algorithm. *Condition* provides the facility for selected processing based on some logical occurrence, and *repetition* allows for looping. These three constructs are fundamental to *structured programming*—an important component-level design technique.

The structured constructs were proposed to limit the procedural design of software to a small number of predictable logical structures. Complexity metrics (Chapter 23) indicate that the use of the structured constructs reduces program complexity and thereby enhances readability, testability, and maintainability. The use of a limited number of logical constructs also contributes to a human understanding process that psychologists call *chunking*. To understand this process, consider the way in which

you are reading this page. You do not read individual letters but rather recognize patterns or chunks of letters that form words or phrases. The structured constructs are logical chunks that allow a reader to recognize procedural elements of a module, rather than reading the design or code line by line. Understanding is enhanced when readily recognizable logical patterns are encountered.

Any program, regardless of application area or technical complexity, can be designed and implemented using only the three structured constructs. It should be noted, however, that dogmatic use of only these constructs can sometimes cause practical difficulties.

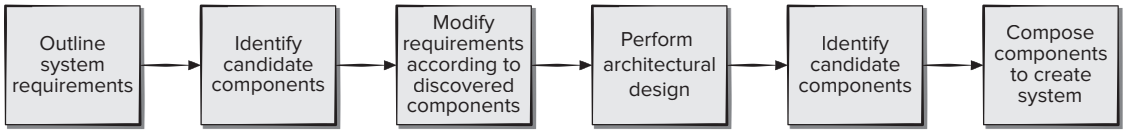
11.4.4 Component-Based Development

In software engineering, reuse is an idea both old and new. Programmers have reused ideas, abstractions, and processes since the earliest days of computing, but the early approach to reuse was ad hoc. Today, complex, high-quality computer-based systems must be built in very short time periods and demand a more organized approach to reuse.

Component-based software engineering (CBSE) is a process that emphasizes the design and construction of computer-based systems using reusable software components (Figure 11.10). Considering this description, many questions arise. Is it possible to construct complex systems by assembling them from a catalog of reusable software components? Can this be accomplished in a cost- and time-effective manner? Can appropriate incentives be established to encourage software engineers to reuse rather than reinvent? Is management willing to incur the added expense associated with creating reusable software components? Can a library of components necessary to accomplish reuse be created in a way that makes it accessible to those who need it? Can existing components be found by those who need them? Increasingly, the answer to each of these questions is yes.

Figure 11.10 shows the principle steps in CBSE. You start with the system requirements and refine them to the point that needed components can be identified. The developers would then search the repository to see if any of the components already exist. Each component has its own postconditions and preconditions. Components whose postconditions match a system requirement are identified, and the preconditions of each component are checked. If the preconditions are satisfied, the component is selected for inclusion in the current build. When no components can be selected, the developers must decide whether to modify the requirements or modify a component that most closely matches the original requirements. This is often an iterative process that continues until the architecture design can be implemented, using a combination of existing or newly created components.

FIGURE 11.10 **Component-based software design**



Consider the task of developing autonomous vehicles, either in real life or a video game. The software for these complex systems is typically created by combing several reusable components, where the components provide distinct modular services. Typically they would include many software components: a component that manages obstacle detection, a planning or navigation component, an artificial intelligence component to manage decision making, and a component of some type controlling vehicle movement or braking. Because these types of software modules have the potential to be used in many different vehicles, it would be desirable to be able to house them in a library of components.

Because CBSE makes use of existing components, it can shorten development time and increase quality. Practitioners [Vit03] often attribute the following advantages to CBSE:

- **Reduced lead time.** It is faster to build complete applications from a pool of existing components.
- **Greater return on investment (ROI).** Sometimes savings can be realized by purchasing components rather than redeveloping the same functionality in-house.
- **Leveraged costs of developing components.** Reusing components in multiple applications allows the costs to be spread over multiple projects.
- **Enhanced quality.** Components are reused and tested in many different applications.
- **Maintenance of component-based applications.** With careful engineering, it can be relatively easy to replace obsolete components with new or enhanced components.

Use of components in CBSE is not without risks. Several of these include the following [Kau11]:

- **Component selection risks.** It is difficult to predict component behavior for black-box components, or there may be poor mapping of user requirements to the component architectural design.
- **Component integration risks.** There is a lack of interoperability standards between components; this often requires the creation of "wrapper code" to interface components.
- **Quality risks.** Unknown design assumptions made for the components makes testing more difficult, and this can affect system safety, performance, and reliability.
- **Security risks.** A system can be used in unintended ways, and system vulnerabilities can be caused by integrating components in untested combinations.
- **System evolution risks.** Updated components may be incompatible with user requirements or contain additional undocumented features.

One of the challenges facing widespread component reuse is *architectural mismatch* [Gar09a]—incompatibilities between assumptions made about components and their operating environments.[7] These assumptions often focus on the component

7 This can be a result of several forms of coupling that should be avoided whenever possible.

control model, the nature of the component connections (interfaces), the architectural infrastructure itself, and the nature of the construction process.

Early detection of architectural mismatch can occur if stakeholder assumptions are explicitly documented. In addition, the use of a risk-driven process model emphasizes the definition of early architectural prototypes and points to areas of mismatch. Repairing architectural mismatch is often very difficult without making use of mechanisms like wrappers or adapters.[8] Sometimes it is necessary to completely redesign a component interface or the component itself to remove coupling issues.

11.5 COMPONENT REFACTORING

Design concepts such as abstraction, hiding, functional independence, refinement, and structured programming, along with object-oriented methods, testing, and software quality assurance (SQA) all contribute to the creation of software components that will be easier to refactor. Most developers would agree that refactoring components to improve quality is a good practice. It is often hard to convince management that it is important to expend resources fixing components that are working correctly instead of adding new functionality to them.

In this book, we focus on the incremental design and delivery of system components. Although there is no quantifiable relationship describing the effects of code changes on architectural quality, most software engineers agree that over time large numbers of changes to a system can lead to the creation of problematic structures in the code base. Failing to address these problems increases the amount of technical debt (Chapter 9) associated with the software system. Reducing this technical debt often involves architectural refactoring, which is generally perceived by developers as both costly and risky. You cannot simply break up large components into smaller components and expect to see an automatic increase in cohesion and a reduction in coupling that will reduce technical debt.

Large software systems may have thousands of components. Making use of data-mining techniques to identify refactoring opportunities can be very beneficial to this work. Automated tools can analyze the source code of system components and make refactoring recommendations to developers, based on generic design rules known to be associated with architectural problems. But it is still up to the developers and their managers to decide which changes to accept and which to ignore [Lin16].

It turns out that many of the error-prone components in a software system are architecturally connected to one another. These flawed architectural connections tend to propagate defects among themselves and accumulate high maintenance costs. If it was possible to automatically identify the technical debt present in the system and the associated maintenance costs, it would be easier to convince developers and managers to spend time refactoring these components. Accomplishing this type of

8 An *adapter* is a software device that allows a client with an incompatible interface to access a component by translating a request for service into a form that can access the original interface.

work requires examining the change histories of the system components [Xia16]. For example, if two or three components are always checked out of the code repository for modification at the same time, it may suggest the components share a common design defect.

11.6 SUMMARY

The component-level design process encompasses a sequence of activities that slowly reduces the level of abstraction with which software is represented. Component-level design ultimately depicts the software at a level of abstraction that is close to code.

Three different views of component-level design may be taken, depending on the nature of the software to be developed. The object-oriented view focuses on the elaboration of design classes that come from both the problem and infrastructure domain. The traditional view refines three different types of components or modules: control modules, problem domain modules, and infrastructure modules. In both cases, basic design principles and concepts that lead to high-quality software are applied. When considered from a process viewpoint, component-level design draws on reusable software components and design patterns that are pivotal elements of component-based software engineering.

Several important principles and concepts guide the designer as classes are elaborated. Ideas encompassed in the open-closed principle and the dependency inversion principle, along with concepts such as coupling and cohesion, guide the software engineer in building testable, implementable, and maintainable software components. To conduct component-level design in this context, classes are elaborated by specifying messaging details, identifying appropriate interfaces, elaborating attributes, and defining data structures to implement them, describing processing flow within each operation, and representing behavior at a class or component level. In every case, design iteration (refactoring) is an essential activity.

Traditional component-level design requires the representation of data structures, interfaces, and algorithms for a program module in sufficient detail to guide in the generation of programming language source code. To accomplish this, the designer uses one of several design notations that represent component-level detail in either graphical, tabular, or text-based formats.

Component-level design for WebApps considers both content and functionality as a Web-based system will deliver it. Content design at the component level focuses on content objects and the ways they may be packaged for presentation to a WebApp end user. Functional design for WebApps focuses on processing functions that manipulate content, perform computations, process database queries, and establish interfaces with other systems. All component-level design principles and guidelines apply.

Component-level design for mobile apps makes use of a multilayered architecture that includes a user interface layer, a business layer, and a data layer. If the mobile app requires the design of components that implement the business and/or data layers on the mobile device, the limitations of the physical characteristics of the device become important constraints on the design.